Timetable Management System
Project Proposal

COURSE : DATABASE SYSTEMS (CS-3006)

INSTRUCTOR : Talha Shahid

GROUP MEMBERS:

Muhammad Bin Ismail (24K-0888)

Safiullah Shaikh (24K-0926)

Jawad Ali (24K-0907)

1. Introduction

Managing academic schedules in a university environment is a complex task. Each semester, administrators must assign hundreds of classes to available rooms, match courses with qualified teachers, and ensure that no two classes conflict in the same room or with the same teacher at the same time. When this is done manually or through basic spreadsheets, errors like double bookings and scheduling overlaps are common.

This project proposes the development of a Timetable Management System — a database-driven web application that automates academic scheduling. The central idea is that correctness and data integrity should be enforced at the database level, not just in application code. This means even if someone bypasses the frontend, the database itself will reject invalid data.

2. Problem Statement

University timetable management faces several recurring issues:

- The same room can accidentally be booked for two different classes at the same time.
- A teacher can be assigned to two different courses during the same time slot.
- There is no automatic record of who changed what and when in the schedule.
- Getting useful reports like teacher workload or room usage requires manual counting.
- Admins have no quick way to check whether a course has been scheduled or not.

These problems can be solved by building a proper relational database with constraints, triggers, and views that enforce rules automatically and generate useful insights without requiring manual queries.

3. Proposed Solution

We propose a full-stack web application with a MySQL database at its core. The system will allow administrators to manage all entities involved in scheduling — departments, teachers, courses, rooms, time slots, and semesters — and will provide a visual timetable interface along with reporting tools.

The key distinction of this system is that database-level mechanisms will handle the hard rules. Triggers will automatically block double bookings. Views will power reports without writing repetitive queries. Stored procedures will handle common lookups. Transactions will ensure that a timetable entry is either fully saved or not saved at all.

4. Technical Stack

Frontend: React with Tailwind CSS for a responsive, modern UI.

Backend: Node.js with Express framework for the REST API.

Database: MySQL 8.0 with raw SQL queries via the mysql2 driver (no ORM).

Reason for no ORM: Using raw SQL gives us full control and clearly demonstrates DBMS concepts like joins, views, and procedure calls.

5. Database Design Overview

The schema will consist of nine tables organized around the central timetable table:

- department — stores faculty departments
- teacher — stores teacher records linked to a department
- course — stores course records linked to a department
- teacher_course — a junction table mapping which teachers can teach which courses
- room — stores room information including type (lecture or lab) and capacity
- time_slot — stores available time blocks per day of the week
- semester — stores semester records linked to departments
- timetable — the main scheduling table linking all entities together
- timetable_audit — logs every insert and update to the timetable with JSON snapshots

The design follows third normal form. Foreign key constraints will be set with ON UPDATE CASCADE and ON DELETE RESTRICT where appropriate to maintain referential integrity.

6. Advanced Database Features

6.1 Triggers

Two BEFORE INSERT triggers will prevent double booking — one checks if the room is already used in that time slot, and another checks if the teacher is already assigned. Two AFTER INSERT/UPDATE triggers will log changes to the audit table.

6.2 Views

Three views will simplify data retrieval: a full timetable view joining all seven related tables, a teacher workload view aggregating classes per teacher, and a room utilization view calculating occupied hours per room.

6.3 Stored Procedures

Three stored procedures will be created for fetching timetables by semester, fetching a teacher's schedule, and finding free rooms for a given time slot. These will be called directly from the backend API.

6.4 Transactions

Timetable insertions will use database transactions with rollback on failure, ensuring that partial writes never corrupt the data.

7. Application Pages

- Dashboard — summary statistics showing total teachers, courses, rooms, and scheduled entries
- Timetable View — a weekly grid showing the complete schedule
- Add Timetable Entry — a form with real-time conflict warning before submission
- Manage Data — full CRUD for teachers, courses, rooms, and time slots
- Reports — teacher workload and room utilization powered by SQL views
- Conflict Checker — scans the timetable for any scheduling conflicts

8. Work Division

Muhammad Bin Ismail: Database design and implementation — schema, triggers, views, stored procedures, audit logging, seed data, and demonstration of database-level features.

Safiullah Shaikh: Full-stack application development — React frontend, Node.js/Express backend, API design, frontend pages, and system integration.

9. Expected Outcomes

By the end of the project, we expect to deliver:

- A fully functional timetable management web application running locally.
- A MySQL database with all constraints, triggers, procedures, and views operational.
- Demonstrable conflict prevention — the system should reject double bookings at the DB level.
- A working audit trail showing a history of all timetable changes.
- Reports generated entirely from SQL views without any manual counting.

10. Conclusion

The Timetable Management System is a practical and academically meaningful project for a Database Systems course. It goes well beyond basic CRUD operations by placing business logic inside the database using triggers, views, procedures, and constraints. This approach demonstrates a real understanding of relational database design and gives us hands-on experience with the features that make database systems powerful in production environments.